Integração AutoLoad ↔ AutoChecker

Análise técnica do contrato, da autenticação e da cobertura de observabilidade

Elaborado em 21/07/2026 · Fontes: documentação oficial de ambos os produtos (REV. 01) e Sentry org automind-qd (janela de 30 dias)

Sumário executivo

O AutoChecker envia automaticamente motoristas, veículos e seus documentos para o **AutoLoad V4** através de uma integração configurável, criada por um assistente próprio (*wizard*) que monta quatro chamadas encadeadas à API do AutoLoad.

A análise cruzou a documentação oficial dos dois produtos (revisão 21/07/2026) com os dados ao vivo do Sentry (org `automind-qd`, janela de 30 dias, consulta projeto a projeto).

Três conclusões:

🔴 **A integração falha de forma contínua e silenciosa.** O despacho de integração do AutoChecker acumula **4.477 falhas em 30 dias** (5.193 desde 14/04) — é o maior ofensor isolado do worker. O erro devolvido é genérico: `{"Message":"An error has occurred."}`.

🔴 **Os dois lados não enxergam o mesmo evento.** Na mesma janela, o AutoLoad registrou **1 evento** em `POST /v1/Drivers`. A diferença de três ordens de grandeza indica que as chamadas estão sendo rejeitadas **antes** de alcançar o código instrumentado do AutoLoad — ou que o AutoLoad não está capturando a falha.

🟠 **A autenticação da integração é o suspeito principal.** Há um descompasso documentado entre a validade do token que o AutoChecker assume e o modelo de sessão do AutoLoad, com sintoma confirmado no Sentry.

---

1. O contrato entre os produtos

1.1 O que o AutoChecker envia

O AutoChecker expõe um endpoint de configuração rápida — `POST /integrations/wizard/autoload-v4/` — que cria o provedor e **quatro endpoints pré-configurados**, executados em ordem:

#	Chamada do AutoChecker	Finalidade
1	`POST /v1/Drivers`	Cria o motorista
2	`GET /v1/Drivers?code="{{cpf}}"`	Verifica se o motorista já existe
3	`PATCH /v1/Drivers/{{driver_id}}`	Atualiza o cadastro
4	`POST` com `$expand` + `multipart/form-data`	Envia os documentos como anexos

O encadeamento funciona por `response_extraction`: o endpoint 1 extrai o `driver_id` da resposta e o injeta no `shared_context`, que os endpoints 3 e 4 consomem no path da requisição.

Existe também um assistente para o **AutoLoad V3** (`v3-enterprise`). São **dois contratos vivos**, não um — a manutenção de compatibilidade precisa considerar ambos.

1.2 O que o AutoLoad expõe

Cruzando com a documentação da API do AutoLoad (cap. IV), os quatro endpoints existem e são compatíveis:

Chamada esperada	Endpoint documentado no AutoLoad	Confere
`POST /v1/Drivers`	`POST /v1/Drivers` — cria motorista	✅
`GET /v1/Drivers` com filtro	`GET /v1/Drivers` — lista com filtro e paginação	✅
`PATCH /v1/Drivers/{id}`	`PATCH /v1/Drivers/{id}` — atualiza	✅
Anexos	`POST /v1/Drivers/{id}/{attributeId}/attachments`	✅

Observação de modelagem: o AutoLoad **não possui uma entidade nativa de documento de motorista**. Os documentos do AutoChecker aterrissam em `DriverAttributeAttachment` — ou seja, como **anexo de atributo customizado**. Isso funciona, mas significa que a inteligência documental do AutoChecker (tipo, validade, status de autenticação) chega ao AutoLoad como metadado de atributo, não como um campo de primeira classe.

---

2. O descompasso de autenticação

Este é o ponto onde as duas documentações não fecham.

2.1 O que o AutoLoad oferece

Caminho	Características
API Key (`UserKey`)	`POST /v1/auth/login` com `{key}`. Possui `IsActive`, `Expiration` própria e flag `IsKeyAccessEnabled`. **É o caminho correto para integração máquina-a-máquina.**
Login de usuário	E-mail + senha. Gera JWT com validade de **24 horas**.

O detalhe crítico está no login de usuário. Segundo o capítulo VII do AutoLoad:

- A sessão (`UserSession`) expira em **30 minutos** — não em 24 horas.
- O parâmetro `same_user_login`, se configurado como `false`, faz com que **um novo login invalide a sessão anterior**.
- O middleware `JwtSingleSessionExtensions` intercepta o token e devolve **401** com a mensagem *"This session has been invalidated by a newer login"*.

2.2 O que o AutoChecker assume

O AutoChecker suporta seis modos de autenticação (`none`, `bearer_token`, `api_key_header`, `api_key_query`, `basic_auth`, `oauth2_client_credentials`) e executa a tarefa `integration_reauth` **a cada 5 minutos** para renovar tokens próximos do vencimento, guiada pelo campo `auth_token_ttl` do provedor.

2.3 O sintoma no Sentry

Issue `AUTOCHECKER-WORKER-8V`, marcada como **escalating**, 70 ocorrências:

```
[ReAuth] Token EXPIRADO para provider 'AutoLoad Cloud'
(expirou em 2026-07-15 13:35:00). Tentando renovar.
```

E, na mesma janela, a própria rotina de renovação falhou:

```
InternalError: cannot execute UPDATE in a read-only transaction
  em shared.tasks.integration_tasks.integration_reauth  (75 ocorrências)
```

Ou seja: **a renovação do token foi tentada, mas o token renovado não pôde ser gravado** porque o banco de dados estava em modo somente-leitura naquele momento.

> 🟠 **Hipótese a validar (não confirmada):** se a integração estiver autenticando como **usuário** em vez de **API Key**, o JWT de 24 h que o AutoChecker acredita possuir na prática morre em 30 minutos pela expiração da sessão — e qualquer login concorrente do mesmo usuário derruba a integração inteira. Verificável inspecionando `auth_type` e `auth_config` do provedor `AutoLoad Cloud`.

---

3. A assimetria de observabilidade

Comparando os dois lados do **mesmo canal**, na **mesma janela de 30 dias**:

Lado	Sinal no Sentry
AutoChecker — `process_integration_event`	**4.477 falhas**
AutoLoad — `POST /v1/Drivers`	**1 evento**

As 15 issues mais frequentes do projeto `autoloadv4-backend` são de áreas totalmente distintas: `RegulatoryEvents` (5.460 eventos, tabela inexistente), filtros dinâmicos em `GET /v1/programs` (652), `Appointments`, `Contracts`, `CustomAttributes`. **Nenhuma relacionada a** `Drivers`.

3.1 O que a mensagem de erro revela

O AutoChecker recebe:

```
{"Message":"An error has occurred."}
```

Esse **não é o formato de erro do AutoLoad**. A documentação (cap. VII, seção 4.7) descreve o `ExceptionMiddleware`, que padroniza toda falha em JSON com mensagem localizada:

- `ServiceException` → 400 Bad Request, mensagem traduzida
- `PermissionDeniedException` → 403 Forbidden
- Exceção genérica → 500 Internal Server Error

O formato `{"Message":"An error has occurred."}` é o **padrão do ASP.NET para exceção não tratada** — o que sugere que a falha ocorre **fora do alcance do middleware** do AutoLoad: em um proxy reverso, no API Gateway, ou na camada de autenticação, antes de o pipeline instrumentado ser alcançado.

> 🔴 **Consequência prática:** hoje é **impossível correlacionar** uma falha de integração do AutoChecker com o evento correspondente do lado do AutoLoad. Não há propagação de `trace_id` entre os produtos — o AutoChecker faz HTTP puro e o AutoLoad não registra a origem da chamada.

---

4. Outros riscos de contrato

4.1 Limite de tamanho de anexo

Produto	Limite
AutoChecker — envio de documento	**20 MB por arquivo**
AutoLoad — `Attachments:MaxSize`	**"5MB"** (valor default documentado)

Se o valor de produção do AutoLoad estiver no default, **todo documento entre 5 MB e 20 MB falha** no endpoint 4. E falharia com erro 4xx — que, na lógica de retry do AutoChecker, significa **falha imediata, sem nova tentativa**.

O AutoLoad também exige uma whitelist de `AllowedContentTypes`, e a aplicação **não sobe** se ela não estiver configurada.

Ação: confirmar o valor real em produção. É configuração, não código.

4.2 Workflow de aprovação

No AutoLoad, `Driver` e `Vehicle` herdam de `ApprovableEntity` e nascem no estado `PENDING`. As regras documentadas (cap. III, seção 5.9):

- Motoristas devem estar `APPROVED` antes de serem atribuídos a agendamentos.
- Veículos devem estar `APPROVED` antes de serem usados em composições.
- Exige as permissões `driver_approval` e `vehicle_approval`.

Ou seja: **um motorista pode ser criado com sucesso pelo AutoChecker e ainda assim permanecer inutilizável no AutoLoad** até que alguém o aprove manualmente.

O AutoChecker prevê esse cenário — é exatamente o status `sent_pending_approval`. Mas **ninguém monitora essa fila hoje**: não há alerta, painel ou relatório sobre entidades paradas aguardando aprovação.

4.3 Dependência de módulos

O tenant do AutoLoad precisa ter o módulo `vehicles_drivers` habilitado (e `appointments`, que depende dele). Sem o módulo, as permissões `driver_management` e `vehicle_management` sequer existem para aquele tenant — e a chamada falha por **autorização**, não por dados. O erro resultante seria indistinguível, para o AutoChecker, de um problema de credencial.

---

5. Comportamento sob falha — o que acontece hoje

Fluxo documentado do AutoChecker quando o AutoLoad recusa a chamada:

Tipo de erro	Comportamento
HTTP 429, 5xx, timeout	Retry com backoff: 60 s → 120 s → 240 s (máx. 3 tentativas)
HTTP 4xx (exceto 429)	Falha imediata, **sem retry**
Após esgotar as tentativas	`integration_status = failed`; só um retry manual recupera

O retry manual existe (`IntegrationLogViewSet.retry()`), mas é uma ação sob demanda: **alguém precisa saber que falhou e ir lá reprocessar.** Sem alerta configurado, isso não acontece.

Isso explica o padrão observado: falhas acumulando de forma constante desde abril, sem intervenção.

---

6. Recomendações

Em ordem de custo-benefício. As quatro primeiras **não exigem alteração de código**.

#	Ação	Onde	Esforço
1	Verificar `auth_type` do provedor `AutoLoad Cloud` — confirmar se usa **API Key** e não login de usuário	Configuração AutoChecker	Baixo
2	Confirmar `Attachments:MaxSize` em produção no AutoLoad e alinhar com os 20 MB do AutoChecker	Configuração AutoLoad	Baixo
3	Confirmar `same_user_login` no tenant AutoLoad usado pela integração	Configuração AutoLoad	Baixo
4	Criar alerta no Sentry para `process_integration_event` — hoje falha em silêncio	Sentry	Baixo
5	Investigar a origem do `{"Message":"An error has occurred."}` — proxy, gateway ou autenticação	Infra	Médio
6	Propagar `trace_id` do AutoChecker para o AutoLoad e registrá-lo do outro lado	Código (ambos)	Médio
7	Criar visibilidade sobre a fila `sent_pending_approval`	Produto	Médio

---

7. O que permanece em aberto

Registrado com honestidade — são pontos que **não** puderam ser determinados a partir da documentação nem do Sentry:

- O `auth_type` real configurado no provedor `AutoLoad Cloud`.
- O valor de produção de `Attachments:MaxSize` no AutoLoad.

- A causa-raiz do banco do AutoChecker ter entrado em modo somente-leitura (5 a 6 dias antes desta análise).
- Se a mensagem `{"Message":"An error has occurred."}` origina-se do AutoLoad ou de uma camada anterior a ele.
- O volume de entidades atualmente paradas em `sent_pending_approval`.

Fontes desta análise

- `Produtos/Autochecker-documentacao.pdf` — REV. 01, 21/07/2026 (capítulos II, III, IV, V e VI)
- `Produtos/Autoload_V4-documentacao.pdf` — REV. 01, 21/07/2026 (capítulos III, IV, VI e VII)
- Sentry, org `automind-qd` — projetos `autochecker-worker`, `autochecker-backend`, `autoloadv4-backend`, `autoloadv4-edgegateway`; janela de 30 dias; consulta projeto a projeto; filtro `is:unresolved level:error`

Elaborado em 21/07/2026 · Fontes: documentação oficial de ambos os produtos (REV. 01) e Sentry org automind-qd (janela de 30 dias)